Министерство науки и высшего образования Российской Федерации
Институт перспективных технологий и индустриального программирования
Кафедра Индустриального программирования

ОТЧЕТ
по практическим занятиям №2

Тема проекта: «Визуализатор алгоритмов сортировки»

Выполнил:
студент группы ЭФБО-04-25
Терешин Максим Геннадьевич

Проверил:
Афонин Сергей Евгеньевич

2026 г.

Шаг 1. Фиксация темы и границ

1. Тема проекта: Разработка визуализатора алгоритмов сортировки на языке C++, предназначенного для наглядной демонстрации работы алгоритмов (пузырьковой сортировки, быстрой сортировки и сортировки слиянием).

2. Scope:

1. Реализация трёх алгоритмов сортировки: пузырьковой, быстрой и слиянием.
2. Визуализация процесса сортировки в реальном времени (пошаговое отображение изменений элементов массива).
3. Управление процессом: выбор алгоритма, запуск, пауза, возобновление и сброс.
4. Регулировка скорости анимации (от пошагового режима до быстрой прокрутки).
5. Формирование исходных данных: генерация случайного массива заданного размера или ручной ввод.
6. Сбор и вывод статистики: количество сравнений, перестановок и затраченное время.

3. Что не входит в проект:

1. Сетевое взаимодействие (клиент-серверная архитектура, удаленный доступ).
2. Сохранение истории сортировок в базу данных или облачные сервисы.

3. Поддержка сортировки сложных структур данных (строк, объектов) — только числовые массивы.
4. Использование сторонних GUI-фреймворков, скрывающих базовую логику отрисовки (визуализация реализуется базовыми средствами).

4. Ограничения:

- Язык реализации: C++.
- Применение сторонних библиотек и сервисов допустимо, но не должно подменять собой реализацию самих алгоритмов сортировки и базовой логики визуализации.

Шаг 2. Выбор метода сбора требований

1. Выбранные методы: Для сбора требований к визуализатору алгоритмов сортировки используются два взаимодополняющих метода:

- **Интервью с «заказчиком»** — для выявления целей проекта и потребностей пользователей.
- **User Stories** — для формализации функций системы в виде бэклога задач.

2. Обоснование выбора:

- **Интервью** позволяет быстро понять контекст использования программы (учебный процесс), выявить реальные потребности (наглядность, понятность) и отделить их от навязанных технических деталей.
- **User Stories** идеально подходят для итерационной разработки, помогают описать функционал с точки зрения пользователя и сформировать понятные критерии приёмки для каждой функции.

3. Как применялись методы:

Метод интервью: В ходе интервью с условным заказчиком были заданы следующие ключевые вопросы:

- *Какова главная цель приложения?* (Ответ: визуальная демонстрация работы алгоритмов для обучения).
- *Какие алгоритмы обязательно нужно реализовать?* (Ответ: базовые — пузырёк, быстрая, слиянием).
- *Как пользователь будет задавать данные?* (Ответ: генерация случайного массива и ручной ввод).
- *Нужна ли статистика?* (Ответ: да, важно видеть количество операций и время).
- *Как должно выглядеть управление?* (Ответ: пауза, скорость, сброс — для детального разбора каждого шага).

Метод User Stories: На основе ответов из интервью были составлены пользовательские истории (подробный бэклог будет в Шаге 4).

4. Результат применения методов:

- Собраны бизнес-цели (зачем нужен продукт).
- Определены пользовательские сценарии.
- Сформирован перечень функциональных возможностей (от выбора алгоритма до вывода статистики).
- Зафиксированы нефункциональные ограничения (плавность анимации, отклик интерфейса).

Вот готовый текст для **Шага 3**. Оформляем в виде таблицы, как того требует методология Вигерса.

Шаг 3. Сбор требований по категориям (по Вигерсу)

3.1. Бизнес-требования (БТ)

ID	Описание требования
БТ-1	Предоставить студентам наглядный инструмент для изучения принципов работы алгоритмов сортировки.
БТ-2	Снизить порог понимания темы «Алгоритмы и структуры данных» за счёт визуализации абстрактных операций.
БТ-3	Дать возможность сравнивать алгоритмы по количеству затрачиваемых операций (сравнений и перестановок) и времени.

3.2. Пользовательские требования (ПТ)

ID	Описание требования
ПТ-1	Пользователь должен иметь возможность наблюдать за процессом сортировки в реальном времени.
ПТ-2	Пользователь должен иметь возможность выбирать один из трёх алгоритмов (пузырёк, быстрая, слиянием).
ПТ-3	Пользователь должен управлять ходом визуализации (ставить на паузу, продолжать, менять скорость, сбрасывать).
ПТ-4	Пользователь должен задавать исходные данные (генерировать случайный массив или вводить вручную).
ПТ-5	Пользователь должен видеть метрики эффективности алгоритма (время, число сравнений и перестановок).

3.3. Функциональные требования (ФТ)

ID	Описание требования
ФТ-1	Система должна генерировать массив случайных чисел заданного пользователем размера.
ФТ-2	Система должна принимать массив чисел, введённый пользователем вручную, и валидировать его.
ФТ-3	Система должна выполнять сортировку выбранным алгоритмом.
ФТ-4	Система должна отображать текущее состояние массива графически (в виде столбцов/элементов) на каждом шаге.
ФТ-5	Система должна обрабатывать команды паузы, возобновления, изменения скорости и сброса в процессе сортировки.
ФТ-6	Система должна подсчитывать количество операций сравнения и перестановки элементов.
ФТ-7	Система должна замерять время выполнения алгоритма от запуска до полной сортировки.
ФТ-8	Система должна выводить итоговую статистику по завершении работы алгоритма.

3.4. Нефункциональные требования (НФТ)

ID	Описание требования
НФТ-1	Плавность визуализации: не менее 30 кадров в секунду при размере массива до 100 элементов.

НФТ-2	Время отклика интерфейса на команды пользователя (пауза, сброс и т.д.) — не более 100 мс.
НФТ-3	Кроссплатформенность: приложение должно собираться и работать на ОС Windows и Linux.
НФТ-4	Корректность: результат сортировки должен совпадать с эталонным отсортированным массивом.
НФТ-5	Архитектура: код должен быть разделён на модули (алгоритмы, визуализация, контроллер).
НФТ-6	Надёжность: система должна корректно обрабатывать некорректный ввод (буквы вместо цифр, пустой массив).

Шаг 4. User Stories (Пользовательские истории)

US-1. Выбор алгоритма

- **История:** Как пользователь, я хочу выбирать один из трёх алгоритмов сортировки (пузырёк, быстрая, слиянием), чтобы изучить их различия.
- **Критерии приёмки:**
  - Доступны ровно 3 алгоритма.
  - Выбранный алгоритм подсвечивается в интерфейсе.
  - Смена алгоритма сбрасывает текущий прогресс.
- **Приоритет:** Must.

US-2. Настройка исходных данных

- **История:** Как пользователь, я хочу генерировать случайный массив или вводить числа вручную, чтобы тестировать алгоритмы на разных данных.
- **Критерии приёмки:**
  - Можно задать размер массива (от 5 до 100 элементов).
  - Генератор создаёт числа в заданном диапазоне.
  - При ручном вводе система проверяет, что введены только числа.
- **Приоритет:** Must.

US-3. Визуализация процесса

- **История:** Как пользователь, я хочу видеть анимацию перестановок элементов, чтобы понимать логику работы алгоритма.
- **Критерии приёмки:**
  - Элементы массива отображаются в виде столбцов разной высоты.
  - Текущие сравниваемые/переставляемые элементы выделяются цветом.
  - Анимация обновляется с частотой не менее 30 FPS.
- **Приоритет:** Must.

US-4. Управление процессом

- **История:** Как пользователь, я хочу ставить сортировку на паузу, менять скорость и сбрасывать прогресс, чтобы детально разбирать каждый шаг.
- **Критерии приёмки:**
  - Кнопка «Пауза» мгновенно останавливает анимацию.
  - Регулятор скорости меняет задержку между шагами.

- Кнопка «Сброс» возвращает массив в исходное состояние.

- **Приоритет:** Must.

US-5. Просмотр статистики

- **История:** Как студент, я хочу видеть количество сравнений, перестановок и время работы, чтобы оценить эффективность алгоритма.
- **Критерии приёмки:**
  - Счётчики сравнений и перестановок увеличиваются в реальном времени.
  - По завершении выводится итоговое время в миллисекундах.
- **Приоритет:** Should.

US-6. Сравнение алгоритмов

- **История:** Как пользователь, я хочу запустить два алгоритма на одинаковых массивах и сравнить их метрики, чтобы понять, какой из них быстрее.
- **Критерии приёмки:**
  - Система сохраняет исходный массив перед первым запуском.
  - Метрики первого алгоритма фиксируются, затем запускается второй.
  - По завершении выводится таблица сравнения (время и операции).
- **Приоритет:** Could.

Шаг 5. Техническая документация (SRS)

1. Введение

1.1. Цель Настоящий документ представляет собой спецификацию требований к программному обеспечению (SRS) для проекта «Визуализатор алгоритмов сортировки». Документ описывает назначение системы, её функции, нефункциональные характеристики, а также служит основой для проектирования архитектуры и дальнейшей разработки.

1.2. Область действия (Scope) Программа представляет собой десктопное приложение на языке C++, предназначенное для визуализации работы алгоритмов сортировки. Система поддерживает пузырьковую сортировку, быструю сортировку и сортировку слиянием. Продукт позволяет генерировать массивы данных, управлять процессом сортировки (пауза, скорость, сброс) и собирать статистику (время, количество операций). Система не включает сетевые функции, базы данных и не предназначена для сортировки сложных структур данных (строк или объектов).

1.3. Определения и аббревиатуры

- **SRS** (Software Requirements Specification) — спецификация требований к программному обеспечению.
- **UML** (Unified Modeling Language) — унифицированный язык моделирования.
- **БТ** — бизнес-требования.
- **ПТ** — пользовательские требования.
- **ФТ** — функциональные требования.
- **НФТ** — нефункциональные требования.
- **FPS** (Frames Per Second) — количество кадров в секунду.
- **User Story** — пользовательская история (формат описания требований в Agile).

2. Общее описание

2.1. Описание продукта Визуализатор алгоритмов сортировки — это учебный инструмент, который превращает абстрактный процесс сортировки чисел в наглядную анимацию. Программа отображает массив в виде столбцов, показывает процесс сравнения и перестановки элементов, а также ведёт подсчёт затраченных ресурсов (времени и операций).

2.2. Функции продукта

- Выбор одного из трёх алгоритмов сортировки.
- Генерация случайного массива или ввод данных вручную.
- Пошаговая визуализация процесса сортировки.
- Управление воспроизведением (пауза, возобновление, изменение скорости, сброс).
- Сбор и отображение статистики (количество сравнений, перестановок, время выполнения).
- Сравнение эффективности двух разных алгоритмов.

2.3. Характеристики пользователей

- **Студенты** (основная аудитория): изучают алгоритмы, используют визуализацию для понимания логики работы.
- **Преподаватели**: используют приложение для демонстрации работы алгоритмов на лекциях и практических занятиях.
- **Уровень подготовки пользователей**: от начального (базовое владение ПК) до продвинутого.

3. Детальные (Специфические) требования

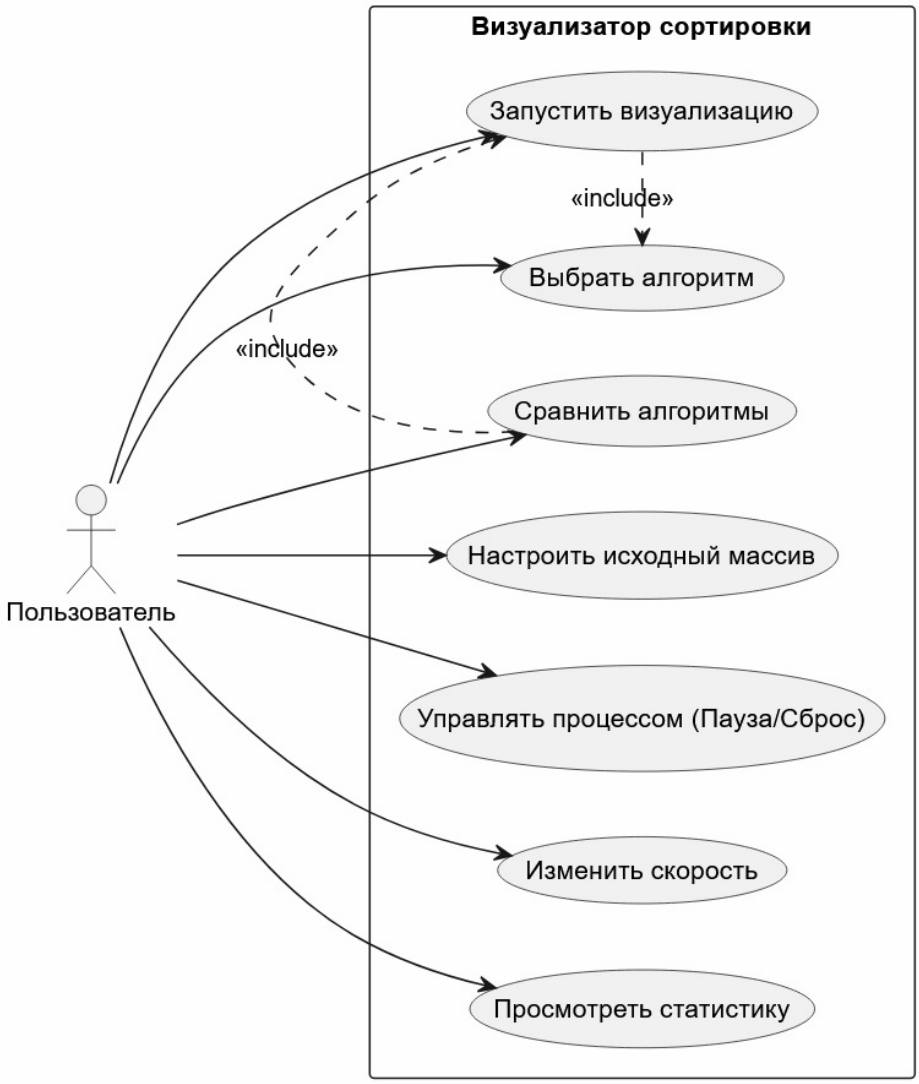
3.1. Использованная методология сбора требований Для сбора требований использовались два метода:

1. **Интервью с условным заказчиком** — помогло определить бизнес-цели (обучение студентов) и ключевые пользовательские сценарии (потребность в наглядности и управлении скоростью).
2. **User Stories (Agile-подход)** — позволило формализовать функциональные требования в виде бэклога задач с критериями приёмки (Must/Should/Could).

3.2. Требования по категориям (по Вигерсу) - **Шаг 3**

Шаг 6. Диаграммы UML

1. Диаграмма вариантов использования (Use Case Diagram)



Что отражает: функциональные требования и роли (акторов).

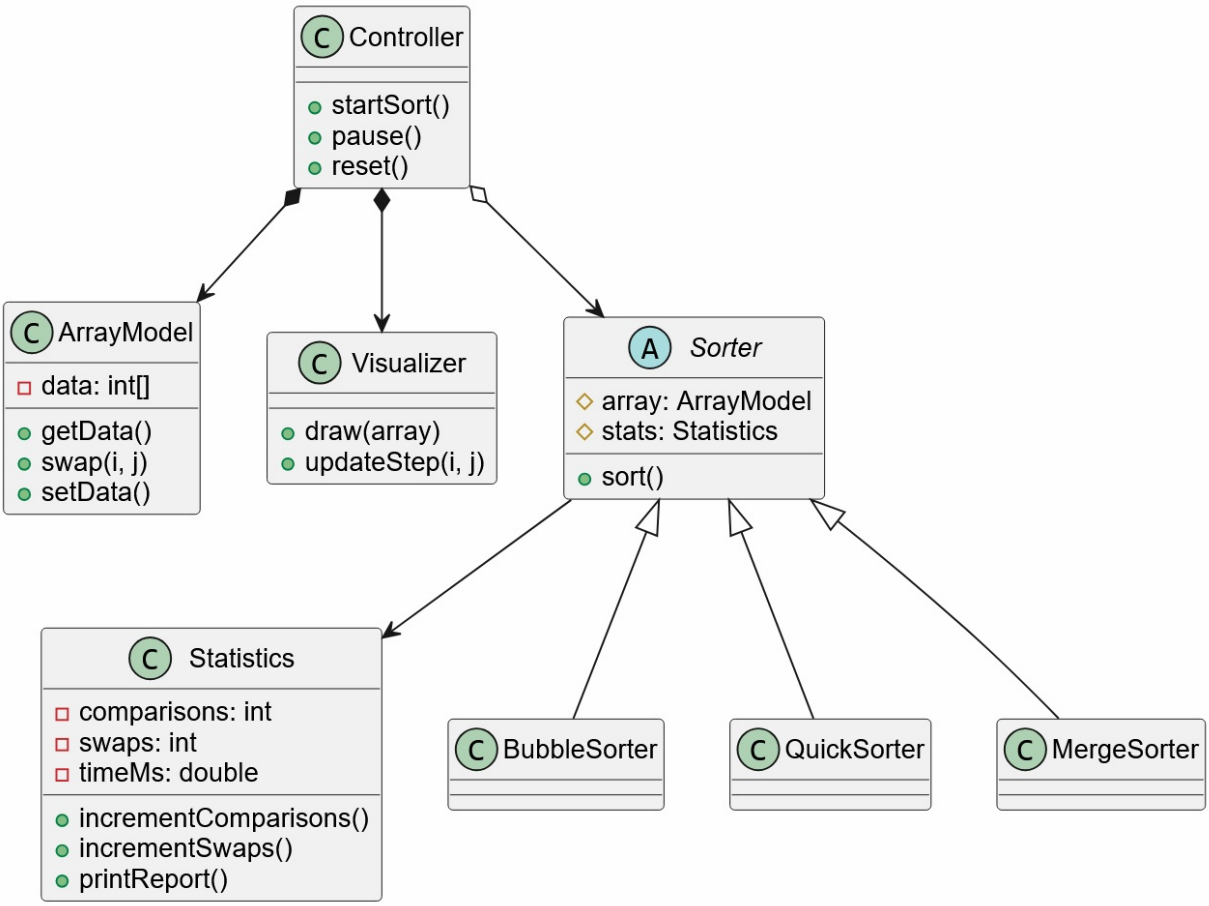
Акторы:

- **Пользователь**.

Прецеденты (Use Cases):

- Настроить массив (сюда входит генерация случайного и ручной ввод).
- Выбрать алгоритм.
- Запустить визуализацию сортировки (включает в себя Паузу и Сброс).
- Изменить скорость.
- Просмотреть статистику.
- Сравнить алгоритмы.

2. Диаграмма классов (Class Diagram)

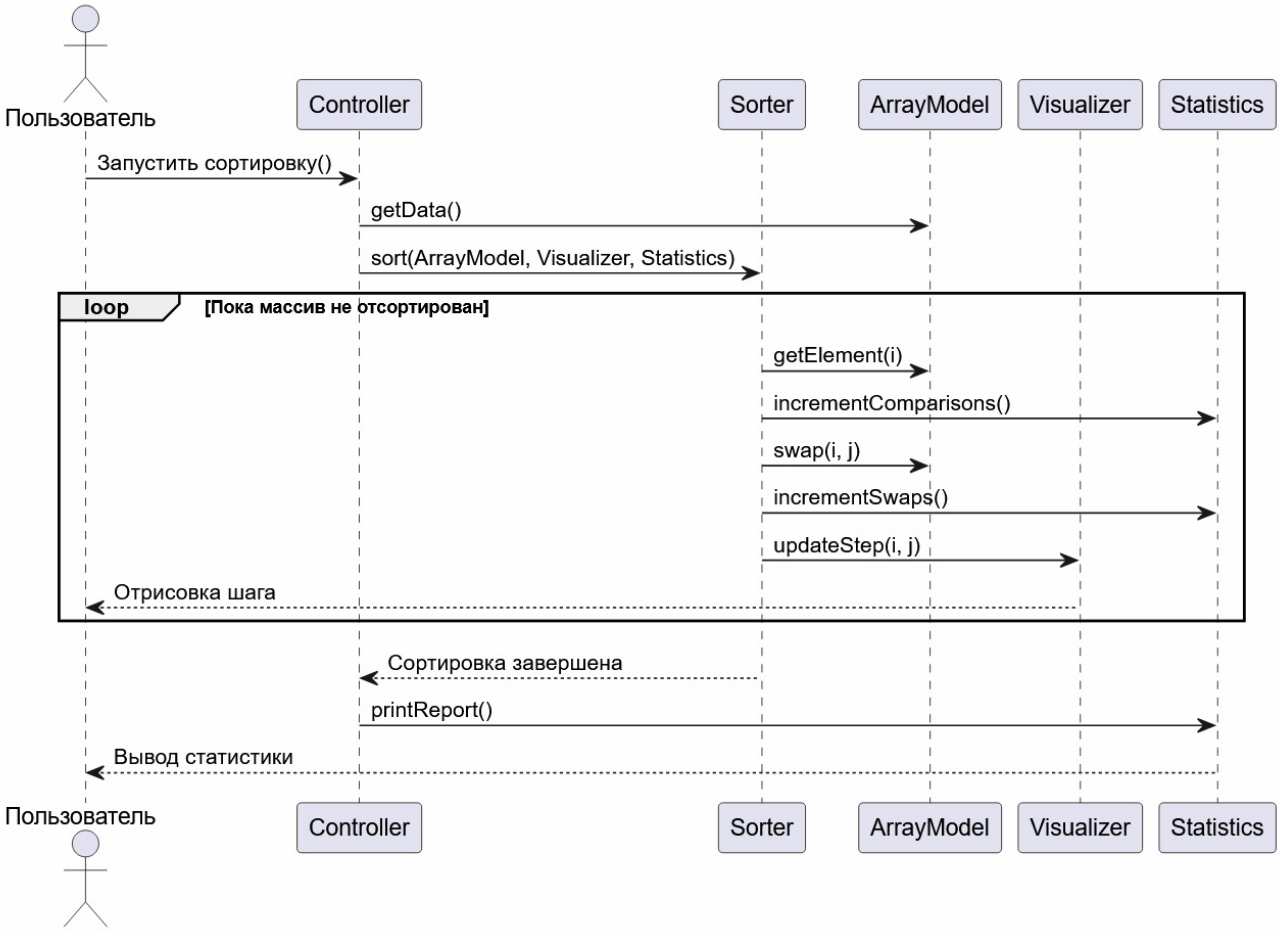


Что отражает: статическую структуру системы, основные сущности и связи между ними.

Основные классы:

- `ArrayModel` (хранит массив чисел, управляет доступом к элементам).
- `Sorter` (абстрактный класс с методом `sort()`).
- `BubbleSorter`, `QuickSorter`, `MergeSorter` (наследуются от `Sorter`).
- `Visualizer` (отвечает за отрисовку массива на экране).
- `Statistics` (собирает метрики: сравнения, перестановки, время).
- `Controller` (связывает всё вместе, обрабатывает действия пользователя).

3. Диаграмма последовательности (Sequence Diagram)



Что отражает: один основной сценарий — **«Запуск процесса сортировки»** (как объекты взаимодействуют во времени).

Участники (объекты): Пользователь, Controller, ArrayModel, Visualizer, Sorter, Statistics.

Сценарий (порядок сообщений):

1. Пользователь нажимает «Запуск» (Controller).
2. Controller обращается к ArrayModel за исходным массивом.
3. Controller передаёт массив и ссылки на Visualizer/Statistics в выбранный Sorter.
4. Sorter начинает цикл: запрашивает элементы у ArrayModel, сравнивает.
5. Если нужна перестановка — Sorter говорит ArrayModel.swap().
6. Sorter отправляет данные в Statistics (плюс сравнение/перестановка).
7. Sorter просит Visualizer обновить экран (отрисовка текущего шага).
8. По завершении Controller запрашивает у Statistics итоговый отчёт и выводит его.